

享内存的进程是不受影响的。

8.3.2 基于共享内存的进程间通信

在简单 IPC 方案中，我们已经看到共享内存是如何帮助两个进程进行通信的。基于共享内存的进程间通信的一个特点是操作系统在通信过程中不会干预数据传输。从操作系统的角度来看，共享内存为两个（或多个）进程在它们的（虚拟）地址空间中映射了同一段物理内存。进程基于这段共享内存设计通信方案，操作系统通常不参与后续的通信过程。

这一小节将介绍基于共享内存的进程间通信的一个经典问题——生产者 - 消费者问题。即存在两个进程，一个是生产者进程，负责产生新的信息，一个是消费者进程，负责消耗信息。生产者和消费者需要共享一块缓冲区（共享内存的用户态抽象）。

代码片段 8.11 给出了该问题中基于共享内存通信的数据结构。其中，buffer 对应的是共享缓冲区。这里我们给定了缓冲区的大小，即 BUFFER_SIZE 个元素。缓冲区中的每个元素由一个结构体 item 表示，这个结构体对应着消息的抽象。

代码片段 8.11 共享内存通信的实现：共享数据结构。这个数据结构和简单 IPC 中的设计类似，但是扩展了缓冲区的大小，并且发送者和接收者也不会固定在特定的缓冲区序号上

```

1 #define BUFFER_SIZE 10
2 typedef struct {
3     struct msg_header header;
4     char data[0];
5 } item;
6
7 item buffer[BUFFER_SIZE];
8 volatile int buffer_write_cnt = 0;
9 volatile int buffer_read_cnt = 0;
10 volatile int empty_slot = BUFFER_SIZE;
11 volatile int filled_slot = 0;

```

有了共享的缓冲区 buffer 之后，生产者的任务就是在缓冲区上产生新的数据。在代码片段 8.12 中，empty_slot 和 filled_slot 分别表示当前在缓冲区上空置的和包含消息的区域的个数。而 buffer_write_cnt 则表示生产者放置新消息的位置。生产者会首先判断当前是否存在空闲的缓存区域，如果有，则写入一个消息，并且对应地修改 empty_slot、filled_slot 和 buffer_write_cnt。注意，这里只考虑单个生产者和单个消费者的情况，对于多线程等情况将在后续章节（同步原语）中展开介绍。

代码片段 8.12 共享内存通信的实现：生产者

```

1 // 生成新的消息
2 int send(item msg)
3 {
4     while (empty_slot == 0)

```

```

5      ; // 没有空闲缓冲区
6      empty_slot--;
7      buffer[buffer_write_cnt] = msg; // msg 的类型是 item
8      buffer_write_cnt = (buffer_write_cnt + 1) % BUFFER_SIZE;
9      filled_slot++;
10     ...
11 }

```

消费者的任务是从缓冲区中获取数据。类似地，在代码片段 8.13 中，通过检查 `filled_slot` 的值，消费者可以判断是否有未处理的消息。对于消费者而言，使用另一个变量 `buffer_read_cnt` 来表示下一个可以读取消息的位置。`recv` 函数会从缓冲区中取出下一个未处理的消息，并对应地修改 `filled_slot`、`empty_slot` 和 `buffer_read_cnt` 的值。

代码片段 8.13 共享内存通信的实现：消费者

```

1 item recv(void)
2 {
3     ... //msg 的类型是 item
4     while (filled_slot == 0)
5         ; // 没有未处理消息
6
7     filled_slot--;
8
9     // 将一个消息从缓冲区中移出
10    msg = buffer[buffer_read_cnt];
11    buffer_read_cnt = (buffer_read_cnt + 1) % BUFFER_SIZE;
12    empty_slot++;
13    return msg;
14 }

```

8.4 消息接口 IPC：消息队列

本节主要知识点

- 消息队列中的消息抽象是怎么设计的？
- 为什么消息队列需要支持类型？

为什么需要消息队列？相比本章介绍的其他通信机制，消息队列是唯一以消息为（内核提供的）数据抽象的通信方式。应用可以通过消息队列来发送消息以及接收消息。发送和接收的接口是内核提供的。此外，消息队列是一种非常灵活的通信机制，支持多个发送者和接收者同时存在。Linux 还为消息队列中的每个消息提供了类型的抽象，使得消息的发送者和接收者可以根据类型来选择性地处理消息。本节将介绍 System V 消息队列。

8.4.1 消息队列的结构

消息队列在内核中的表示是队列数据结构，如图 8.8 所示。当创建新的消息队列时，内核将从系统内存中分配一个队列数据结构，作为消息队列的内核对象。可以看到，这个对象有其对应的权限，以及消息头部指针。队列的消息由这个头部指针引出，每个消息都有指向下一个消息的指针（或者为空）。这是一种常见的队列的链表设计。在消息的结构体中，除了“下一个”指针之外，就是消息的内容。消息的内容包含两部分：数据和类型。数据是一段内存数据，和管道中的字节流相似。类型是用户态程序为每个消息指定的。在消息队列的设计中，内核不需要知道类型的语义，仅仅保存并基于类型进行简单的查找。如图 8.8 所示，第一个消息的类型是 1。这可能代表消息中的数据是一个字符串，也可能代表该数据是一个结构体。类型的具体意义需要用户态程序自己来管理。

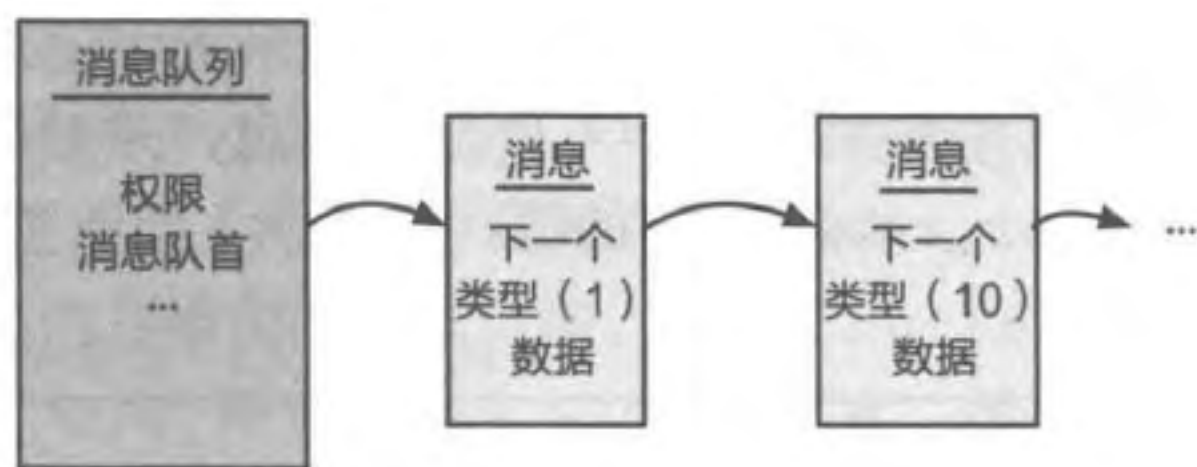


图 8.8 消息队列

8.4.2 基本操作

消息队列的操作一般被抽象为四个基本操作：`msgget`、`msgsnd`、`msgrcv`、`msgctl`。这四个操作在 Linux 系统上被实现为系统调用。`msgget` 允许进程获取已有消息队列的连接，或者创建一个新的消息队列。消息队列本质上采用的是信箱的通信方式。发送者和接收者在通信过程中，只需要建立好对同一个信箱（此处的“队列”）的访问，就相当于建立了通信连接。只要有对应的权限，消息队列允许任意数量的进程连接到同一通信连接上（即同一队列上）。`msgctl` 可以控制和管理消息队列，如修改消息队列的权限信息或删除该消息队列。

进程可以通过 `msgsnd` 向消息队列发送消息，通过 `msgrcv` 从消息队列接收消息。多个进程可以同时向队列发送消息或从队列接收消息。消息的发送和接收成功的标志，是消息被放到队列上或者从队列上取出。大部分情况下这两个过程是非阻塞的：对发送者来说，只要队列有空闲的空间就可以向队列发送消息，而接收者只要有未读消息就可以直接读取消息并完成操作。若发送消息时消息队列没有可用空间或接收消息时没有未读消息，默认的操作是阻塞进程，直到有空间腾出或者新的消息到来。Linux 内核在发送和接收消息的接口（`msgsnd` 和 `msgrcv`）上允许用户指定是否等待（`NOWAIT` 选项）。若用户指定了 `NOWAIT`，内核可以直接返回相关的错误信息，用户进程不会阻塞。

练习

8.8 消息队列是否支持类似超时机制的特性？请解释理由。

8.5 案例分析：L4 微内核的 IPC 优化

本节主要知识点

- L4 针对进程间通信展开了哪些优化？
- L4 的数据传递（或消息传递）机制是怎样设计的？

早期微内核 Mach 的复杂性对进程间通信的性能影响很大。根据 Mach 的经验，Liedtke 等研究人员开始研发 L4 系列的微内核系统^[1-4]。L4 系列微内核系统设计的一个突出思路是：进程间通信是微内核的核心功能，需要围绕通信完成整个系统的设计和实现。L4 是当下仍然十分主流的微内核系统，特别是后续衍生出了各种变体和相关的系统。图 8.9 给出了 L4 系统从 1993 年开始的一些发展脉络^[8]。本节内容将以 L4 早期 IPC 设计为主，同时介绍相关设计在后续系统中的使用和优化。

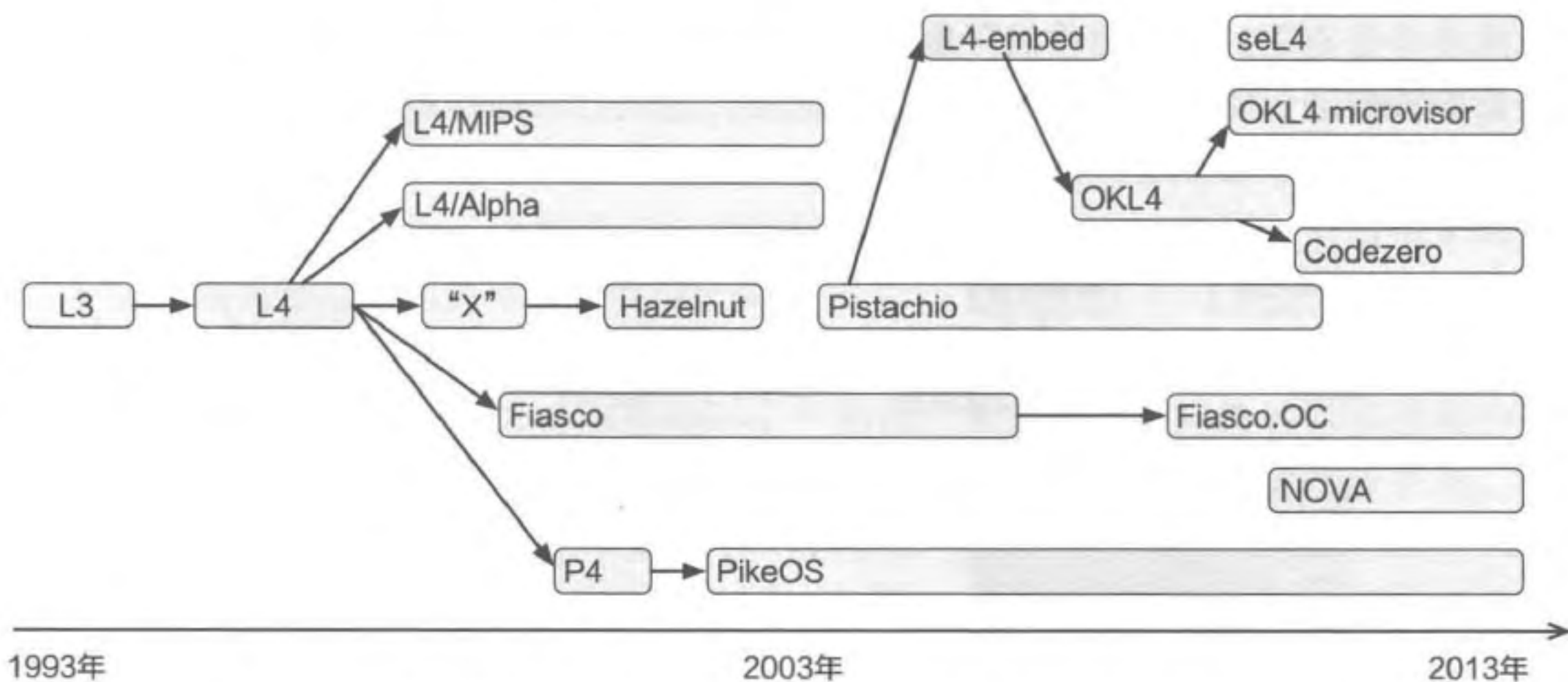


图 8.9 L4 微内核家族及其演进，简化自研究者 Gernot Heiser 的总结^[8]

在 L4 微内核中，内核只保留了基本的功能，包括地址空间、线程、进程间通信等，并且不考虑兼容性等要求，而是选择针对特定硬件做极致的性能优化。这样做的好处是内核的代码量非常少，可以为少量的功能提供尽可能完善的支持。下面将从消息传递^①、控制流转移、通信连接和权限检查四个方面介绍 L4 中 IPC 的设计。

8.5.1 L4 消息传递

在 L4 早期的设计中，为了尽可能减少内核暴露的接口，给单个通信接口增加了丰富的语义。这一点主要体现在传递的消息上：除了类似函数调用的消息外，还支持几乎

① 由于 L4 的设计中均使用消息接口，因此我们直接使用“消息传递”来替代“数据传递”。